Introduction to LAMMPS - A Guide to Molecular Dynamics Simulations

Author: Samah Gourari
Master's student in Computational Physics, University of Tlemcen, Algeria, with research in physics-informed neural networks (PINNs) and computational modeling.

SECTION 1: Introduction to Molecular Dynamics and LAMMPS

Molecular Dynamics:

As physicists, questioning the phenomena surrounding us is the foundation of scientific research. To understand nature, we must first understand motion: why particles move, how they interact, and how their collective behavior emerges. This understanding allows us to predict physical properties and develop reliable simulation tools, such as Molecular Dynamics.

Imagine wanting to predict and track the evolution of a system composed of a huge number of particles: a material being stretched or compressed, water molecules diffusing inside a container, the response of a tiny metal sample under stress to

understand crack formation, or even the folding of a protein.

The laws governing particle motion are well known, but determining the behavior and properties of such complex systems is analytically impossible, simply because tracking the motion of every particle individually is not feasible.

The solution to studying systems at this scale is computational analysis. Since we are dealing with nanometer length scales and time scales ranging from femtoseconds to microseconds, Molecular Dynamics (MD) is the appropriate simulation method. MD allows us to explicitly follow the motion of particles by numerically solving their equations of motion over time.

Molecular Dynamics is a computational technique based on classical mechanics, unlike ab-initio methods which rely on quantum mechanics. The central idea is simple: the force acting on each particle is calculated using Newton's second law,
$F = m \cdot a$.
A molecular dynamics simulation begins with a system composed of a large number of particles, to which initial positions and velocities are assigned.

To compute the forces between particles, MD relies on an interatomic potential. A potential is a mathematical description of how the energy of a system depends on the positions of its atoms. Intuitively, atoms tend to evolve toward configurations of minimum potential energy. Since force is given by the negative derivative of the potential energy, defining a potential allows us to directly compute the forces acting on each particle.

Interatomic potentials are empirical or semi-empirical models designed to approximate atomic interactions. They generally arise from a balance between repulsive forces at short distances and attractive forces at longer distances. When atoms move, oscillate, and exchange energy, Newtonian forces drive them toward equilibrium states. In Molecular Dynamics, choosing an appropriate potential is therefore essential to correctly describe attraction, repulsion, and the overall system behavior.

There exists a large variety of interatomic potentials, each developed to specific systems and phenomena. For example, the Lennard-Jones potential is commonly used for simple fluids and

noble gases, while metallic systems often require many-body potentials such as the Embedded Atom Method (EAM). We have other potentials such as Tersoff potentials for covalent materials, ionic potentials for charged systems, and more complex force fields for molecular and biological systems. The choice of potential depends on what system is being studied and which properties interest us..

Once the initial positions, velocities, and interatomic potential are defined, the simulation proceeds by calculating the forces acting on each particle. Using these forces, the system is advanced forward in time by a very small time step, which gives us new velocities and new positions. This process is repeated iteratively, step by step, allowing the system to evolve naturally toward equilibrium (a state where the energy fluctuates around a minimum and macroscopic properties become stable).

Through this process, Molecular Dynamics helps calculate both static and dynamic properties, such as energetic, structural, thermal, mechanical, and diffusion-related properties. However, because MD is based on classical mechanics, it cannot directly

describe electronic, optical, or electromagnetic properties.

But even with these limitations, Molecular Dynamics remains a powerful computational tool for understanding atomic-scale motion. At the very beginning, MD simulations began with only a few atoms treated as hard spheres, but advances in computational power and modeling techniques have expanded its applicability to liquids, solids, and complex materials. Nowadays, MD has a fundamental role in materials science, it helps researchers understand microscopic mechanisms and guide the design of new materials with specific properties.

LAMMPS & MD Simulations:

After understanding how Molecular Dynamics works in principle, a natural question arises:
How do we actually perform these simulations on a computer? How do we tell the machine to follow all these steps including defining particles,

computing forces, integrating equations of motion, and tracking the system over time?

In theory, nothing prevents us from writing a Molecular Dynamics code from scratch. As computational physicists, we often use programming languages such as Python, Fortran, C/C++, or MATLAB, combined with numerical libraries, to define a system, choose a potential, compute forces, and integrate Newton's equations of motion step by step. This approach is extremely instructive and lets us have full control over the simulation, but it quickly becomes complex and time-consuming.

This is where LAMMPS (Large-scale Atomic/Molecular Massively Parallel Simulator) comes in. LAMMPS is a software package specifically designed for Molecular Dynamics simulations, it allows users to perform realistic atomistic simulations without having to implement every numerical detail from scratch. Instead of writing thousands of lines of code, the user describes the system and the simulation workflow through an input script, and LAMMPS takes care of the heavy computational work.

The strength of LAMMPS is its flexibility and scalability. It can run on a single processor for small systems, but it is also designed for parallel execution using MPI and OpenMP and this makes it suitable for large-scale simulations involving millions of particles. LAMMPS supports a wide variety of particle types, force fields, and interaction models, including simple atomic systems, molecular systems, coarse-grained models, and hybrid combinations of different particle types.

LAMMPS can simulate both 2D and 3D systems, handles different boundary conditions, and even accounts for finite-size or complex particle shapes. In addition to Molecular Dynamics, it also includes support for certain Monte Carlo moves, which can be useful for studying specific thermodynamic properties.

LAMMPS also includes the numerical integrators needed to evolve the system in time. An integrator is the algorithm that takes the forces acting on particles and determines their new positions and velocities after a small time step. Intuitively, it answers the question: given where the particles are

now and how strongly they are being pushed, where will they be a moment later.

In this sense, LAMMPS is a reliable engine that implements the core ideas of Molecular Dynamics discussed earlier; forces derived from interatomic potentials, time integration of Newton’s equations,

and statistical sampling of physical properties, all while leaving the user free to focus on the physics of the system, rather than on low-level numerical implementation.

In the following sections, we will explore how LAMMPS works in practice, how to install it, how to write a basic input script, and how to run and analyze a simple Molecular Dynamics simulation.

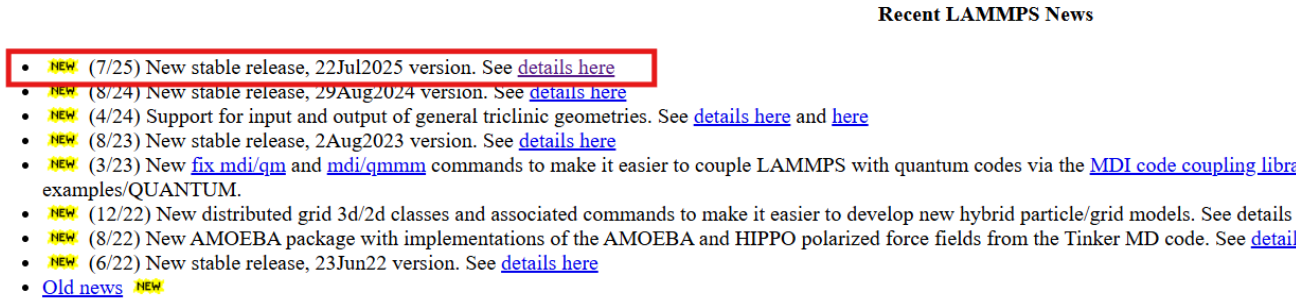
SECTION 2: LAMMPS Installation and Computational Environment

LAMMPS Installation (Windows):

Below is a structured step by step installation guide:

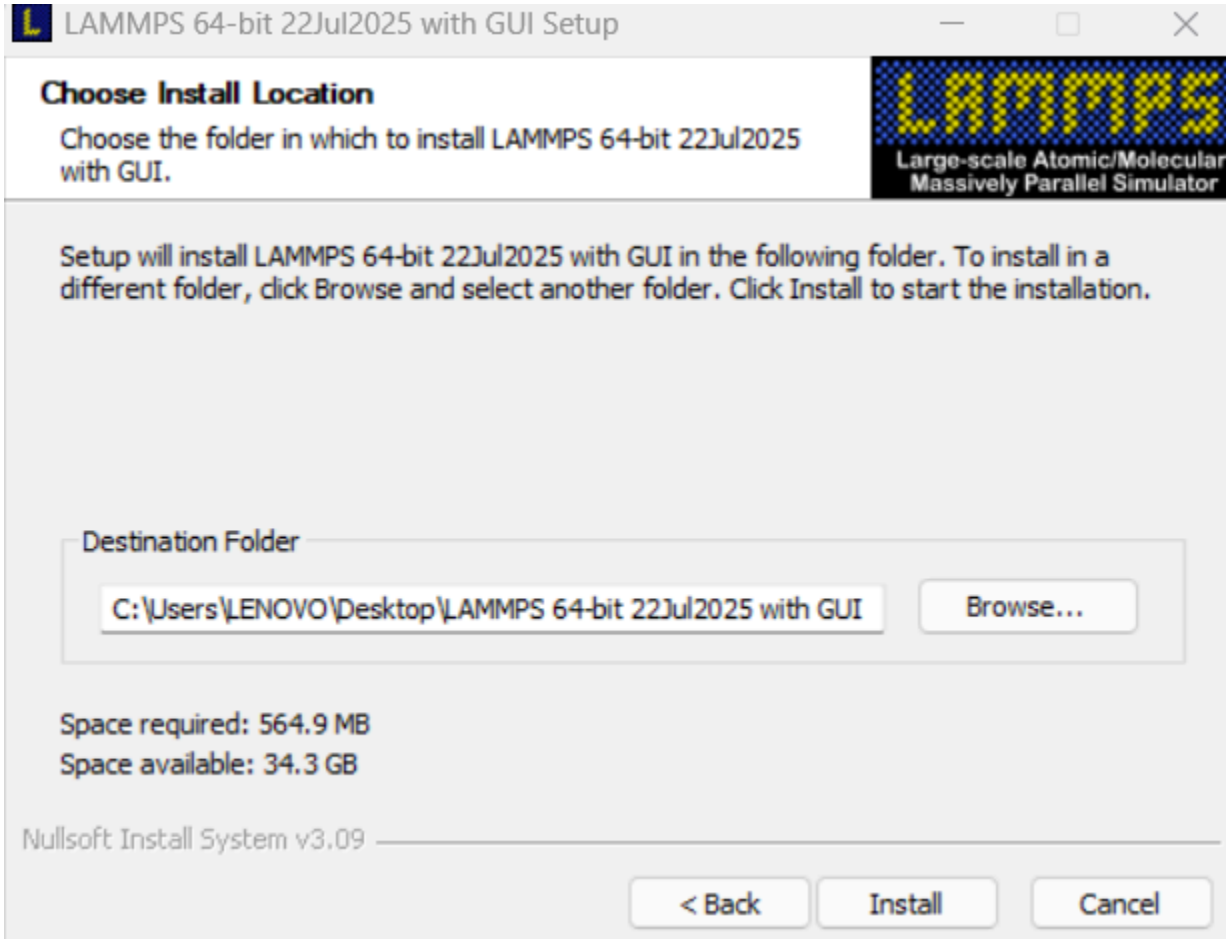
1) Go to https://www.lammps.org/

2) Scroll down to the “Recent LAMMPS News” section and click on the latest stable release, this will redirect you to another page.

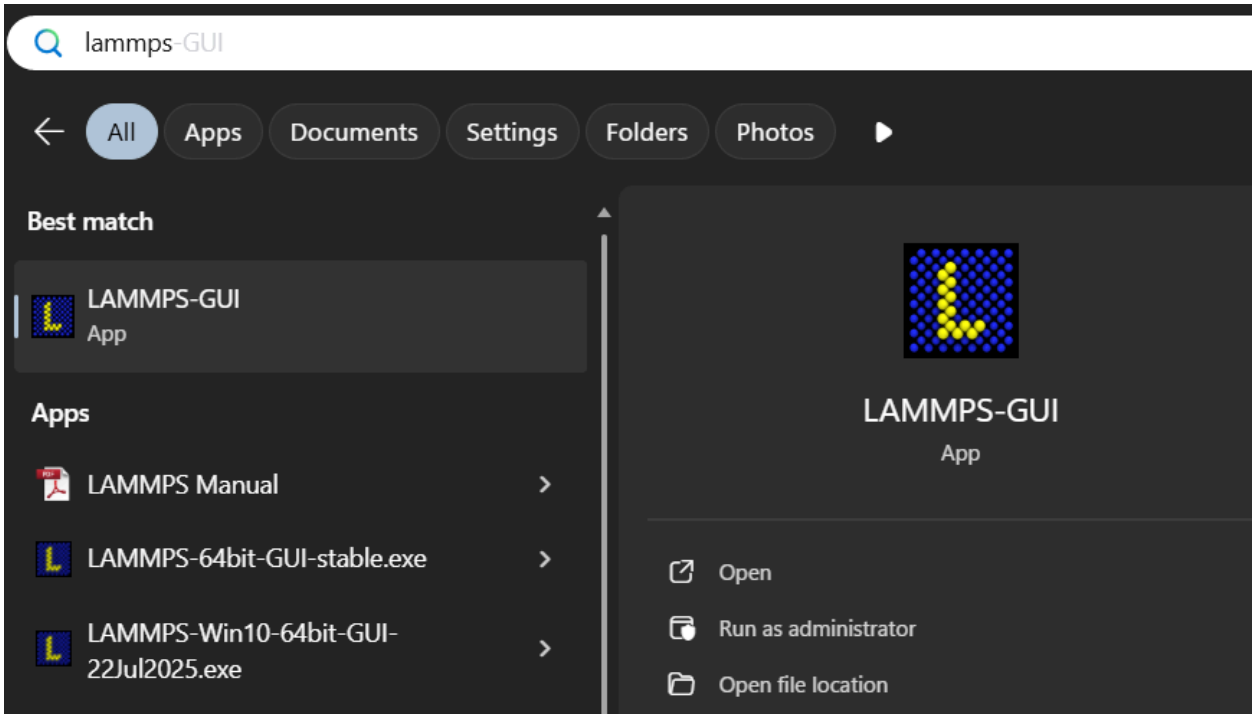


3) The link opens the GitHub page of the latest stable LAMMPS release, which contains release notes and update information. Scroll all the way down to the “Assets” section to find the executable file. Click on it for download.

4) After downloading the executable file, launch it to open the installation window. Accept the license agreement, choose an installation folder (preferably an easily accessible location), and click Install.

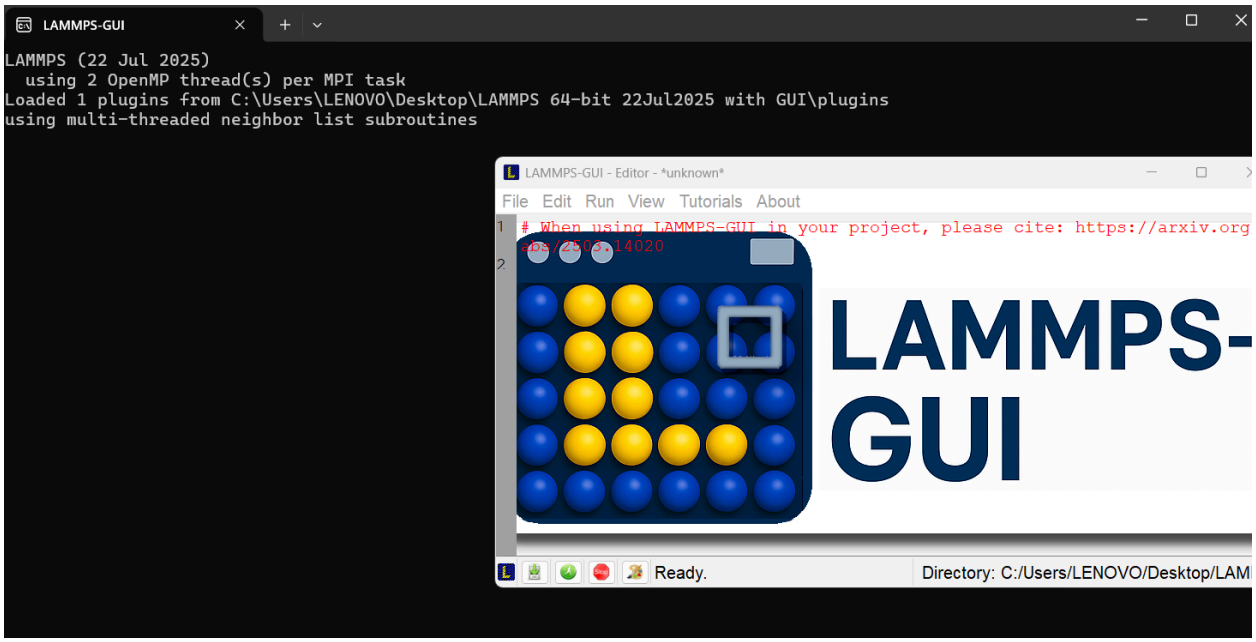


5) Once the installation is complete, close the setup window. Then, open the system's search bar, type LAMMPS-GUI, and click "Open" to start the program.



6) When launching LAMMPS, two windows will appear. The first one is a command-line (terminal) window, shown as a black screen. This window displays technical information about the simulation environment, such as parallel execution settings (OpenMP and MPI), loaded plugins, and system configuration. It runs automatically in the background and does

not require any user interaction for basic simulations. The second window is the LAMMPS graphical user interface (GUI), which is the main working environment. For better visibility, fullscreen the LAMMPS GUI window.

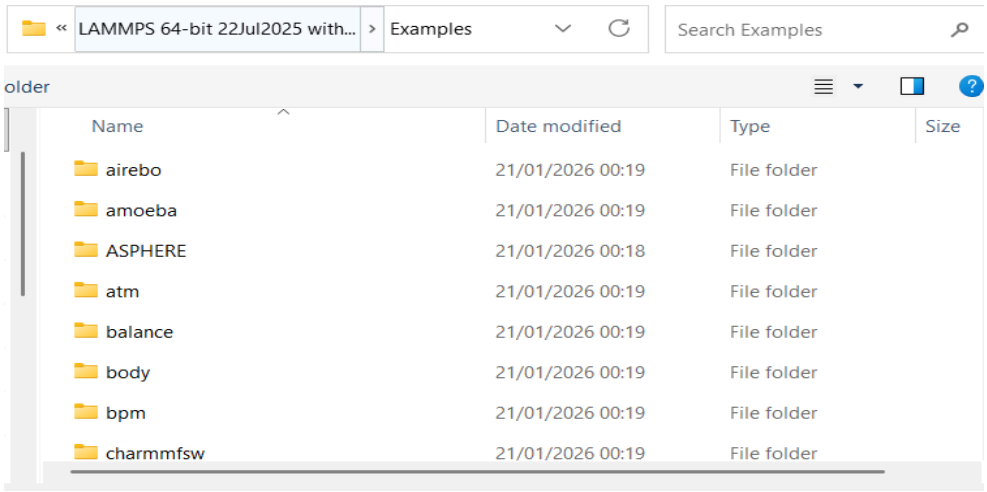


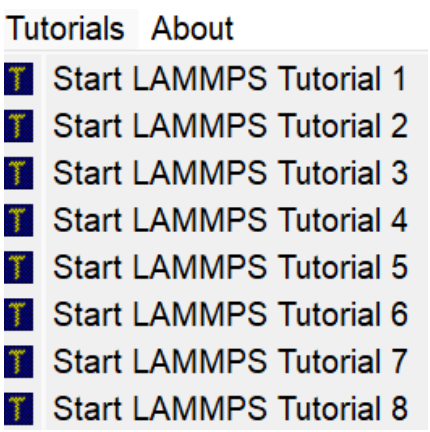
7) After maximizing the window, you will see a first comment line at the top. This is a note from the developers reminding users to cite LAMMPS properly. On the left side you will see line numbers, helping you keep track of your code. The white area is where you can write your own code, defining your system, particles and all other parameters.

If you prefer not to start from scratch, LAMMPS also provides example files that come with the installation. To access them, click File → Open, then navigate to the examples folder inside your LAMMPS installation directory. Those files are pre-written LAMMPS scripts representing various systems, such as stabilized materials, and other molecular systems. They allow you to run ready simulations and modify parameters.

You can also click on Tutorials, which lists around eight examples. Each tutorial links to a webpage showcasing a system with a specific potential. For our case, we'll use Tutorial 1, just click on it to open the page. Next, we'll go through this tutorial step by step and explain it in simple terms.


LAMMPS-GUI - Editor -
File Edit Run View Tutorials About
When using LAMMPS-GUI in your project, please cite: https://arxiv.org/abs/2503.14020
LAMMPS-
GUI
Ready.
Directory:


LAMMPS 64-bit 22Jul2025 with...
Examples
Search Examples
older
Name
Date modified
Type
Size
airebo 21/01/2026 00:19 File folder
amoeba 21/01/2026 00:19 File folder
ASPHERE 21/01/2026 00:18 File folder
atm 21/01/2026 00:19 File folder
balance 21/01/2026 00:19 File folder
body 21/01/2026 00:19 File folder
bpm 21/01/2026 00:19 File folder
charmmfsw 21/01/2026 00:19 File folder


Tutorials
About
Start LAMMPS Tutorial 1
Start LAMMPS Tutorial 2
Start LAMMPS Tutorial 3
Start LAMMPS Tutorial 4
Start LAMMPS Tutorial 5
Start LAMMPS Tutorial 6
Start LAMMPS Tutorial 7
Start LAMMPS Tutorial 8

Computational Environment:

Molecular Dynamics simulations can be very demanding on your computer, especially when dealing with many particles or long simulation times. Your PC's performance, such as CPU speed, number of cores, RAM, and storage, affects how fast simulations run and the size of the system you can study.

To check your system's specifications on Windows:

> → Press Win + R, type `dxdiag`, and hit Enter, this shows your CPU, RAM, and graphics.

Knowing your PC specs helps explain why some simulations run slower or why you might need to reduce the number of particles or the simulation time.

SECTION 3: Step-by-Step Tutorial: Lennard-Jones Fluid Using LAMMPS

Lennard-Jones Fluid:

In this tutorial, we simulate a Lennard-Jones (LJ) fluid. But why start with this model?

A Lennard-Jones fluid is one of the simplest models in MD. It represents a collection of many particles (atoms) confined in a box, where each pair of atoms interacts through the same simple rule: the Lennard-Jones potential. There are no charges, no chemical bonds, and no electronic effects, just neutral particles interacting through distance-dependent forces. For this reason, the Lennard-Jones fluid is often considered a toy model to start with. Think of it as trying to visualize how a liquid behaves inside a box, or how a gas moves inside a closed room.

The LJ potential describes two physical effects:

- Strong repulsion at very short distances, when atoms get too close and their electron clouds

overlap (Pauli repulsion).

- Weak attraction at intermediate distances, caused by temporary fluctuations of electrons that create induced dipoles between nearby atoms (van der Waals).

- Almost no interaction at large distances, where atoms are too far apart to influence each other.

Mathematically, the LJ pair potential is written as:

$$U(r) = 4\varepsilon[(\sigma/r)^{12} - (\sigma/r)^{6}]$$

where:

- r is the distance between two atoms,

- ε is the depth of the potential well (interaction strength).

- σ is the distance at which the potential is zero.

The term r^{12} represents the strong short-range repulsion, while the term r^{6} represents the attractive van der Waals interaction.

From this potential, the force between two atoms is obtained as the negative derivative of the potential:

$$F(r) = -\, dU(r)/dr$$

$$F(r) = 24\varepsilon/\sigma\,[\,2(\sigma/r)^{13} - (\sigma/r)^{7}]$$

This is why defining a potential is central in molecular dynamics: once the potential is known, forces can be computed, and Newton's equation of motion can be solved.

The Lennard-Jones potential is a pair interaction potential, it depends only on the distance between two atoms and it reproduces experimental data such as equilibrium spacing and cohesive energy. While it cannot describe chemical reactions or electronic effects, it is ideal for studying basic fluid behavior.

Now that we understand what kind of system we want to simulate, we can start writing the LAMMPS input script step by step.

Step-by-Step Script with Explanations:

Before going through the code line by line, it is important to note that the LAMMPS input script is divided into 2 main parts.

Part A: Dedicated to energy minimization, in this first part, we set up our simulation "universe." We define the overall dimensions and boundaries, which act like the walls of our universe. Inside this universe, we create a smaller box where we will study our fluid, you can imagine it like a small room inside our vast universe. For practical reasons, we focus only on a small section of this room, and in our case, we choose a cylinder shape because it is convenient and natural for the type of simulation we want to run. After that, we insert the atoms, and specify the interaction parameters. At this point, the atoms are placed in a rather arbitrary way and may overlap or experience unrealistically large forces.

Energy minimization is performed separately to relax the system and remove these artificial stresses. An intuitive way to think about it is like organizing people before a race, at the beginning, everyone arrives chaotically and stands too close to

each other. Before starting the race, they need to spread out and settle into reasonable positions. Energy minimization plays exactly this role, it allows the system to reach a stable, low-energy configuration before we start the actual dynamics.

Part B: The molecular dynamics simulation itself, where we run the system under the influence of specific ensembles like NVE (constant energy if we work with an isolated system) or NVT (constant temperature influence, as if our system is in a heat bath). Here, the atoms begin to move, interact, and evolve over time. This part uses the relaxed structure saved from Part A as the starting point, just like the runners who are now properly lined up and spaced, ready to start the race. The simulation then tracks how the system evolves step by step, to produce the trajectories and properties we want to study.

PART A

1) Initialization

`units lj` → sets the unit system to Lennard-Jones units.

`dimension 3` → defines the simulation in 3D space (X, Y, Z).

`atom_style atomic` → treats each particle as a simple point with mass, no bonds or charges.

`boundary p p p` → applies periodic boundary conditions: if an atom leaves one side of the box, it re-enters from the opposite side, simulating an infinite system and avoiding wall effects.

2) System Definition

`region simbox block -20 20 -20 20 -20 20` → defines the overall simulation box as a cubic region, extending from -20 to 20 in (X, Y, Z). This is our “Universe”.

`create_box 2 simbox` → creates a simulation box that can contain two atom types (type 1 and type 2). This prepares the system for adding different kinds of particles.

`region cyl_in cylinder z 0 0 10 INF INF side in` → defines a cylindrical region along the Z-axis, centered at (0,0), with a radius of 10, which will contain type 2 atoms. This is like selecting a smaller “room within the room” where certain particles will be confined.

`region cyl_out cylinder z 0 0 10 INF INF side out` → defines the area outside the cylinder, where type 1 atoms will be placed. Everything not in “`cyl_in`” is part of “`cyl_out`”.

`create_atoms 1 random 1000 34134 cyl_out` → randomly distributes 1000 atoms of type 1 in the “`cyl_out`” region. The number `34134` is a seed for the random placement to ensure

reproducibility.

`create_atoms 2 random 150 12756 cyl_in` → randomly distributes 150 atoms of type 2 inside the cylinder. The number 12756 is also just another seed for reproducibility.

3) Settings

`mass 1 1.0` → mass of type 1 atoms (1.0). They are light and will move faster.

`mass 2 5.0` → mass of type 2 atoms (5.0). Heavier atoms move slower.

`pair_style lj/cut 4.0` → chooses the LJ potential with a cutoff distance of 4 units. Atoms farther apart than 4 units barely interact, so forces beyond this are ignored.

`pair_coeff 1 1 1.0 1.0` → interaction for type 1–type 1 atoms: ε = 1.0 (strength of attraction) and σ = 1.0 (size of the particle).

`pair_coeff 2 2 0.5 3.0` → interaction for type 2–type 2 atoms: ε = 0.5 (weaker attraction) and σ = 3.0 (larger size).

For type 1–type 2 interactions, LAMMPS calculates it using geometric mixing: $\sigma_{1,2} = \sqrt{(\sigma_{1,1} \cdot \sigma_{2,2})}$, $\varepsilon_{1,2} = \sqrt{(\varepsilon_{1,1} \cdot \varepsilon_{2,2})}$.

4) Monitoring

`thermo 10` → tells LAMMPS to print thermodynamic information every 10 steps.

`thermo_style custom step etotal press` → chooses what information to display:

step = current simulation step,
etotal = total energy of the system,
press = instantaneous pressure.

5) Run and Save

`minimize 1.0e-6 1.0e-6 1000 10000` → performs energy minimization:
first two numbers = energy and force tolerances,
last two numbers = max iterations and max evaluations.
This relaxes the system to remove overlaps and huge forces before dynamics.

`write_data improved.min.data` → saves the minimized configuration into a file named "improved.min.data" for later use in the next simulation (Part B).

Energy minimization finds a local minimum, not necessarily the global one, producing a configuration with lower potential energy, no unrealistic overlaps, and zero kinetic energy. LAMMPS uses the conjugate gradient algorithm, which repeatedly computes forces, moves atoms slightly, and repeats until energy or forces stop changing. The minimization stops when one of four criteria is met: force tolerance, energy tolerance, maximum iterations, or maximum force evaluations. Initially, the potential energy is high due to strong repulsions from overlapping atoms; it quickly drops, then slowly decreases, reaching negative values as atoms settle at reasonable distances. Since this is not dynamics, kinetic energy remains zero, and atoms become less chaotic, more evenly spaced, and loosely clustered.

PART B

Before opening the file from Part A, it’s a good idea to save it in a dedicated folder. This way, all outputs from Part B, like images, log files, and data, will be stored in the same place instead of cluttering your desktop (Trust me, I learned this the hard way, my desktop once looked like a bomb exploded with hundreds of files!). Once the folder is ready, you can open the file in LAMMPS by clicking File → Open and selecting the saved data file

6) Grouping

`group grp_t1 type 1` → selects all type 1 atoms and puts them in group grp_t1.

`group grp_t2 type 2` → selects all type 2 atoms and puts them in group grp_t2.

`group grp_in region cyl_in` → selects all atoms inside the cylinder region and puts them in group grp_in.

`group grp_out region cyl_out` → selects all atoms outside the cylinder region and puts them in group grp_out.

`group grp_t1_in intersect grp_t1 grp_in` → finds atoms that are both type 1 and inside the cylinder.

`group grp_t2_out intersect grp_t2 grp_out` → finds atoms that are both type 2 and outside the cylinder.

`delete_atoms group grp_t1_in` → removes type 1 atoms that are inside the cylinder.

`delete_atoms group grp_t2_out` → removes type 2 atoms that are outside the cylinder.

7) Cleaning

Earlier, we created groups to select atoms by type or region so we could manipulate them easily, for example, deleting atoms in certain regions. Now that we've done those operations, we delete these temporary groups to clean up.

`group grp_in delete` → deletes the temporary group grp_in

`group grp_out delete` → deletes the temporary group grp_out

`group grp_t1_in delete` → deletes the temporary group grp_t1_in

`group grp_t2_out delete` → deletes the temporary group grp_t2_out

8) Variables and Computes

`variable n1_in equal count(grp_t1,cyl_in)` → counts how many type 1 atoms are inside the cylinder

`variable n2_in equal count(grp_t2,cyl_in)` → counts how many type 2 atoms are inside the cylinder

`compute coor12 grp_t1 coord/atom cutoff 2 group grp_t2` → for each atom in group 1, calculates distances to atoms in group 2 within 2 units (like checking neighbors only nearby, ignoring the others)

`compute sumcoor12 grp_t1 reduce ave c_coor12` → averages all the distances computed above to get a single representative value

We calculate the distances and their average to monitor how our two types of atoms are distributed in the cylinder over time. The individual distances (compute coor12) tells us local interactions, which atoms are near each other. The average (compute sumcoor12) gives a single number that summarizes the overall closeness or mixing of the two groups. Later in the simulation, this can be used to track changes: for example, if the average distance decreases, atoms are clustering; if it increases, they are spreading out.

9) Monitoring

`thermo 1000` → prints simulation info every 1000 steps

`thermo_style custom step temp pe ke etotal press v_n1_in v_n2_in c_sumcoor12` → shows step, temperature, potential & kinetic energy, total energy, pressure, number of atoms in cylinder, and average closeness between types

`dump viz all image 1000 myimage-*.ppm type type shiny 0.1 box no 0.01 view 0 0 zoom 1.8 fsaa yes size 800 800` → creates images every 1000 steps all atoms, colored by type, 3D shiny spheres, hide box edges, set view, zoom, resolution

`dump_modify viz adiam 1 1 adiam 2 3 acolor 1 turquoise acolor 2 royalblue backcolor white` → adjusts image details: set atom diameters, color type 1 turquoise, type 2 royal blue, white background

10) Running the MD Simulation

Before starting the simulation, we need to give the atoms some initial motion, because otherwise, in a system at NVE (constant number of atoms, volume, and energy), nothing would move. We assign velocities to all atoms corresponding to a temperature of 1.0 (in LJ units). You can think of this as giving the atoms an initial push, so they start moving naturally.

The Gaussian distribution is just a way to assign random speeds that match the expected distribution of velocities at that temperature, like how real particles move randomly in a fluid. We also set the total momentum to zero to make sure the system doesn't drift in one direction, like if everyone in a crowded room started running in the same way, we want them to jiggle around randomly.

Next, we tell LAMMPS to evolve the system using Newton's equations in an isolated system. Energy, volume, and number of atoms stay constant. This is

like watching a closed box of particles move around without any energy going in or out.

The Langevin thermostat is a small correction we add to control temperature slightly. Even though NVE keeps energy constant, our initial velocity assignment may not give the exact temperature we want.

At the end of it all, we control how long we let the atoms evolve and how precise the motion calculation is. Each timestep is a tiny frame of motion, and the run command repeats this for a definite amount of steps, so we can observe the behaviour of the system. Here, we chose to only simulate NVE, the isolated system, but we could also use NVT or NPT if we wanted to immerse our system in a heat bath at constant temperature or under constant pressure, basically an external influence. For this tutorial, we keep it simple with NVE.

`velocity all create 1.0 49284 mom yes dist gaussian` → initializes velocities for all atoms at temperature 1.0, use random seed 49284, zero total momentum, Gaussian distribution

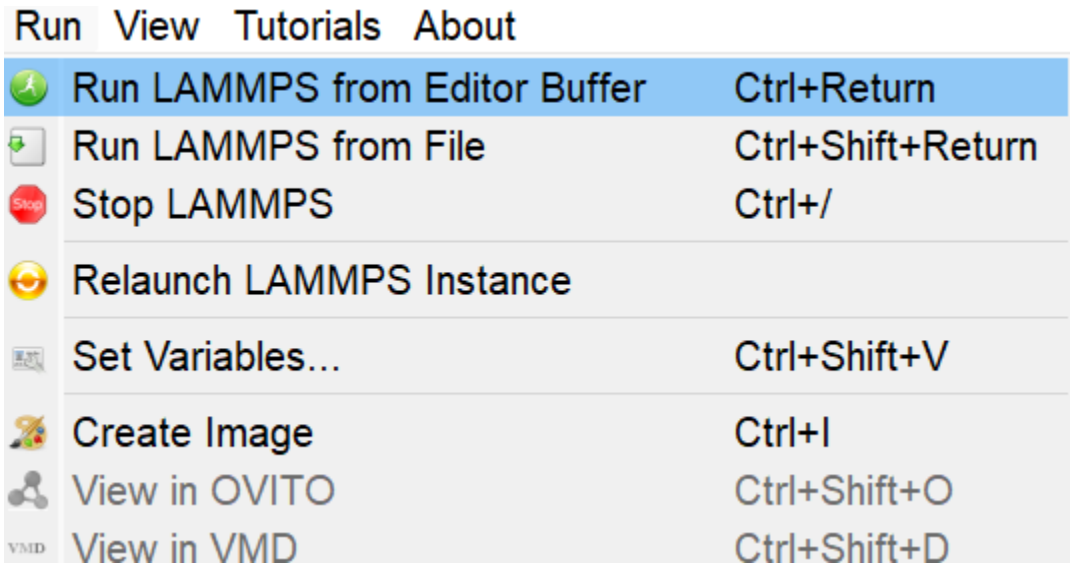
`fix mynve all nve` → integrates equations of motion using NVE ensemble (constant number of atoms, volume, and energy)

`fix mylgv all langevin 1.0 1.0 0.1 10917 zero yes` → applies Langevin thermostat to maintain temperature target 1.0, damping 0.1, random seed 10917, zero linear momentum

`timestep 0.005` → sets simulation time step to 0.005 (LJ units)

`run 300000` → runs the simulation for 300,000 steps

Finally, our system is fully set up, the atoms are placed, grouped, and ready, the interactions are defined, and all simulation parameters are in place. To run the simulation, we do it directly from the editor buffer in LAMMPS, which will start evolving the system. In the next section, we will see the results and interpret them to understand how our LJ fluid evolves over time.



SECTION 4: Execution, Results, Analysis and Interpretation

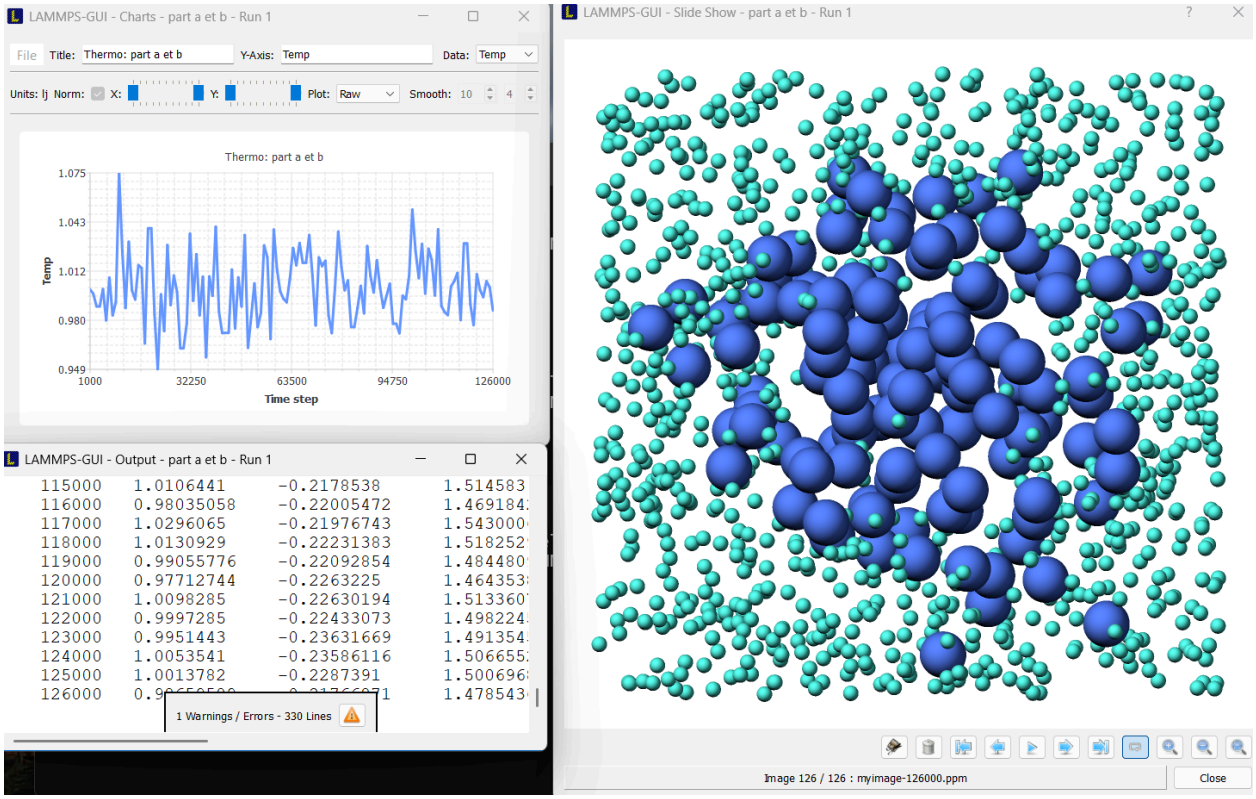
Execution:

LAMMPS starts executing the simulation and several windows appear.

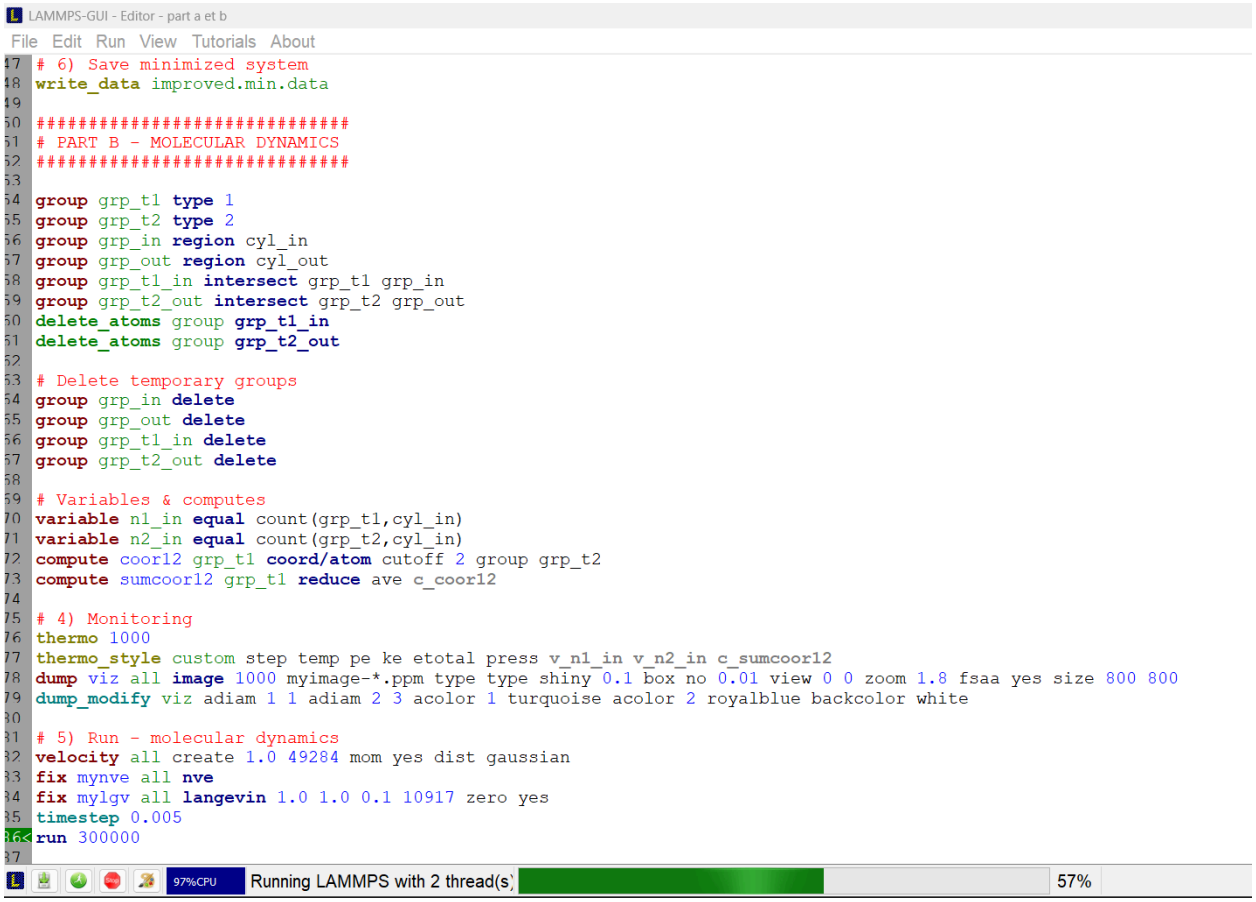
The first window shows the animation of the atoms, allowing us to visually follow how the system evolves with time and how particles rearrange inside the simulation box.

The second window displays a graph showing the temperature as a function of time. This plot allows us to monitor the thermal behavior of the system and verify that the temperature fluctuates around the target value (here close to 1 in LJ units), which is expected for a well-equilibrated system.

The third window contains the numerical output, where all computed quantities such as temperature, energies, pressure, and other variables are printed step by step during the simulation.


LAMMPS-GUI - Charts - part a et b - Run 1
File
Title: Thermo: part a et b
Y-Axis: Temp
Data: Temp
Units: lj Norm:
X:
Y:
Plot: Raw
Smooth: 10 4
Thermo: part a et b
1.075
1.043
1.012
0.980
0.949
Temp
1000
32250
63500
94750
126000
Time step
LAMMPS-GUI - Output - part a et b - Run 1
115000 1.0106441 -0.2178538 1.514583
116000 0.98035058 -0.22005472 1.469184
117000 1.0296065 -0.21976743 1.543000
118000 1.0130929 -0.22231383 1.518252
119000 0.99055776 -0.22092854 1.484480
120000 0.97712744 -0.2263225 1.464353
121000 1.0098285 -0.22630194 1.513360
122000 0.9997285 -0.22433073 1.498224
123000 0.9951443 -0.23631669 1.491354
124000 1.0053541 -0.23586116 1.506655
125000 1.0013782 -0.2287391 1.500696
126000 0.9
1.478543
1 Warnings / Errors - 330 Lines
LAMMPS-GUI - Slide Show - part a et b - Run 1
Image 126 / 126 : myimage-126000.ppm
Close

At the same time, in the code editor interface, we can see at the bottom that LAMMPS is running, with information about the number of threads used (for example, 2 threads in our case on my computer) and a progress bar indicating the percentage of completion of the simulation.



Results:

To analyze the evolution of the system, 2 types of results are considered:

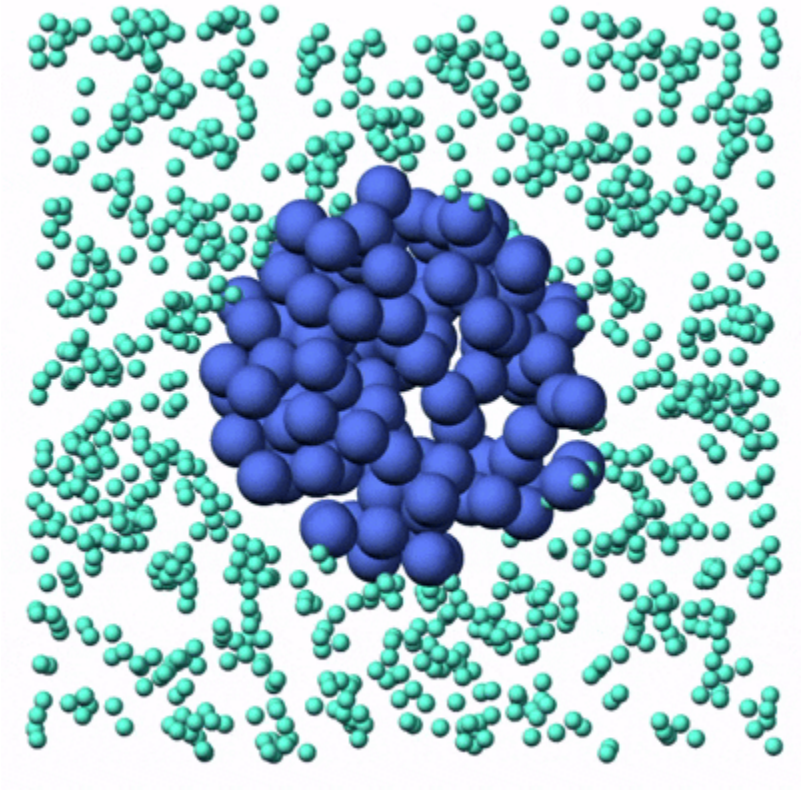
- A visual result: a GIF showing the time evolution of particle positions.

- Quantitative results: thermodynamic and structural quantities extracted from the LAMMPS output during the simulation.

The numerical results include temperature, potential energy, kinetic energy, total energy, pressure, and coordination-related quantities. Due to the large number of timesteps, only representative values are reported here

.

Visual Observation (GIF)



Here we see an initially disordered configuration, particles rearrange due to strong LJ forces. Over time, the system evolves toward a stable state. No particle collapse or unphysical behavior is observed, which indicates a stable integration scheme and appropriate timestep.

Numerical Results

```
Per MPI rank memory allocation (min/avg/max) = 5.032 | 5.032 | 5.032 Mbytes
```

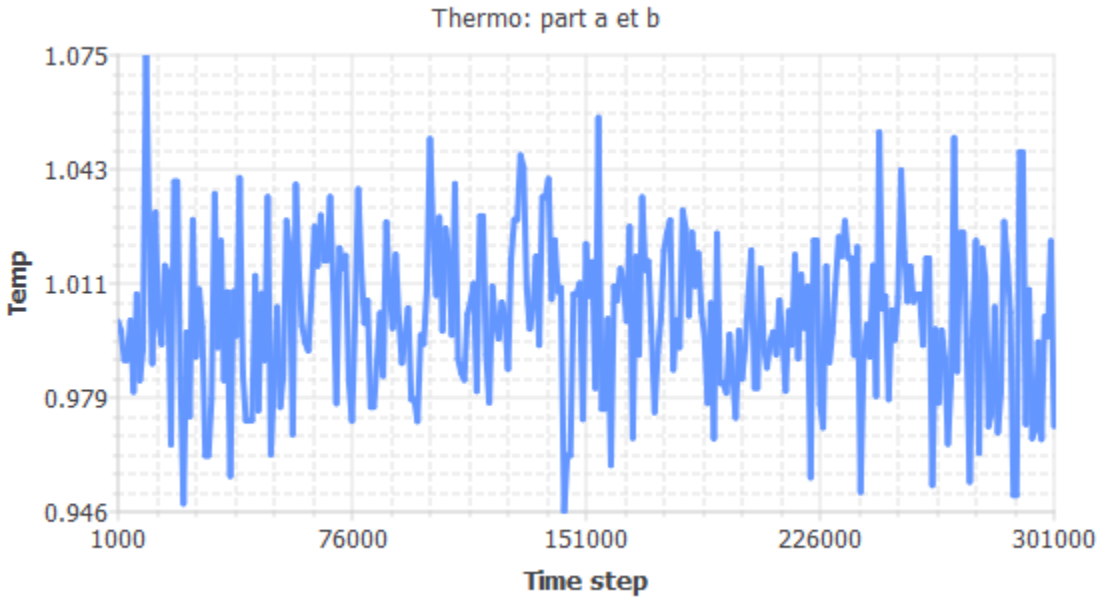
Step	Temp	PotEng	KinEng	TotEng	Press	v_n1_in	v_n2_in	c_sumcoor12
1000	1	-1.2368942	1.4986314	0.26173721	0.017082336	0	126	0.034020619
2000	0.99728018	-0.68433854	1.4945554	0.81021683	0.01872896	2	122	0.013402062
3000	0.98913505	-0.55591069	1.4823488	0.92643815	0.017374407	4	122	0.011340206
4000	0.99449267	-0.52218066	1.4903779	0.96819727	0.017409293	6	121	0.0051546392
5000	1.0002655	-0.4689317	1.4990292	1.0300975	0.015973397	10	118	0.010309278
6000	0.98017365	-0.42720453	1.468919	1.0417145	0.017301255	9	119	0.0082474227
7000	1.0076414	-0.40011835	1.510083	1.1099647	0.015545571	11	121	0.006185567
8000	0.98339712	-0.36929046	1.4737498	1.1044593	0.016500626	10	116	0.0082474227
9000	0.99197167	-0.34637581	1.4865999	1.1402241	0.016538034	8	119	0.0010309278
10000	1.074914	-0.3356378	1.6108999	1.2752621	0.017868161	11	115	0.0020618557
11000	1.0245606	-0.33513761	1.5354387	1.2003011	0.015740365	10	117	0.006185567
12000	0.98811467	-0.31809676	1.4808197	1.1627229	0.015307926	11	115	0.010309278
13000	1.0306769	-0.31123212	1.5446047	1.2333726	0.016539036	10	114	0.010309278
14000	0.99942714	-0.29886815	1.4977729	1.1989047	0.016044062	10	112	0.0072164948
15000	0.99357353	-0.2953135	1.4890005	1.193687	0.015860304	12	112	0.0041237113
16000	1.0156777	-0.27727301	1.5221265	1.2448534	0.017117949	14	114	0.0051546392
17000	1.0138047	-0.26407099	1.5193196	1.2552486	0.017980228	14	112	0.0030927835
18000	0.96532842	-0.26466267	1.4466715	1.1820088	0.01579385	19	114	0.0072164948
19000	1.0393394	-0.27250292	1.5575866	1.2850837	0.017157079	19	113	0.0072164948
20000	1.0331889	-0.26173556	1.5483693	1.2866338	0.017907082	21	112	0.0072164948
21000	0.98397112	-0.25043125	1.47461	1.2241787	0.017390324	20	113	0.0092783505
22000	0.9487288	-0.2495739	1.4217948	1.1722209	0.016433661	23	108	0.0051546392
23000	0.99690733	-0.2369153	1.4939966	1.2570813	0.018093601	27	107	0.013402062
24000	0.97326445	-0.25208138	1.4585646	1.2064833	0.01662409	20	110	0.012371134
25000	1.0285579	-0.2536396	1.5414292	1.2877896	0.016775019	24	109	0.015463918
26000	0.98998829	-0.24441523	1.4836275	1.2392123	0.015525508	22	109	0.0082474227
27000	1.009033	-0.24153306	1.5121685	1.2706354	0.016832602	21	109	0.0082474227
28000	0.99833535	-0.24065815	1.4961367	1.2554785	0.015636205	22	106	0.0092783505
29000	0.96222143	-0.23593724	1.4420152	1.206078	0.015682742	21	102	0.0092783505
30000	1.0014724	-0.24110221	1.500838	1.2597358	0.015043453	20	103	0.010309278
31000	0.9782105	-0.24563669	1.465977	1.2203403	0.015296175	23	100	0.0092783505

This presents selected simulation outputs at different timesteps, spanning the early transient regime and the later quasi-equilibrated regime. All values are reported exactly as produced by LAMMPS.

Analysis:

Temperature Evolution

The temperature fluctuates around the target value of T ≈ 1, with deviations. These fluctuations are expected in molecular dynamics and they showcase the stochastic nature of the Langevin thermostat. More importantly, no systematic drift toward higher or lower temperatures is observed, which indicates that the system has reached and maintained thermal equilibrium.



Energy Evolution

Potential energy (PotEng) increases rapidly from a negative value at early times (−1.2368942 at step 1000) toward less negative values.
This shows the relaxation from a non-equilibrated configuration where particles are too close.
Kinetic energy (KinEng) fluctuates around values compatible with the target temperature.
Total energy (TotEng) stabilizes after the initial transient, with small oscillations.
This behavior indicates proper energy exchange between kinetic and potential components, towards equilibrium.

Pressure Evolution

The pressure remains positive and fluctuates around values of approximately **0.015–0.017** throughout the simulation. These fluctuations are caused by particle collisions, but no large pressure spikes are observed

Coordination Evolution

The coordination-related quantities (v_n1_in, v_n2_in, and c_sumcoor12) show an evolution. At early times, “v_n1_in” is very small (0 at step 1000), while” v_n2_in” is large. As the simulation progresses, “v_n1_in” steadily increases, while “v_n2_in” decreases.
The quantity “c_sumcoor12” increases over time, and reaches values above 0.04.
Particles gradually form more stable local positions.

.

Interpretation:

Remember our race track analogy from earlier ?

At the beginning of the simulation, all runners start at once, but not everyone runs smoothly. Some accelerate too fast, some slow down, some bump into others. The system is adjusting. After a short time, the runners settle into a rhythm, they’re still moving differently, but the overall flow becomes stable. That’s when the system reaches equilibrium.

The temperature is like the average speed of all runners. It never stays perfectly constant because individuals constantly speed up or slow down, but as long as the average speed stays around the same value, the race is well balanced. Those small fluctuations are normal and physical, not errors.

The energy exchange is like runners going uphill and downhill. When a runner climbs (strong interactions), they lose speed (kinetic energy) and gain potential energy, downhill, it's the opposite. Individually, things change all the time, but the total effort of the race stays constant.

The cutoff is runners only paying attention to nearby runners. Someone very far away doesn't affect how you run.

This simple simulation shows a race that starts chaotic, then settles into a steady flow, which is exactly what we expect from a physical system reaching equilibrium, and exactly what we wanted to simulate with Molecular Dynamics.

SECTION 5: Sensitivity of the Simulation to Key Parameters

Now that the simulation runs and produces results, we can experiment by changing a few key parameters to see how the system behaves differently (don't forget to also play with colors!)

Adjusting temperature / thermostat

Temperature controls how much kinetic energy the atoms have. It directly changes diffusion, clustering and whether structures form or break apart. Almost everything you observe depends on it.

What to touch (the " T " below)

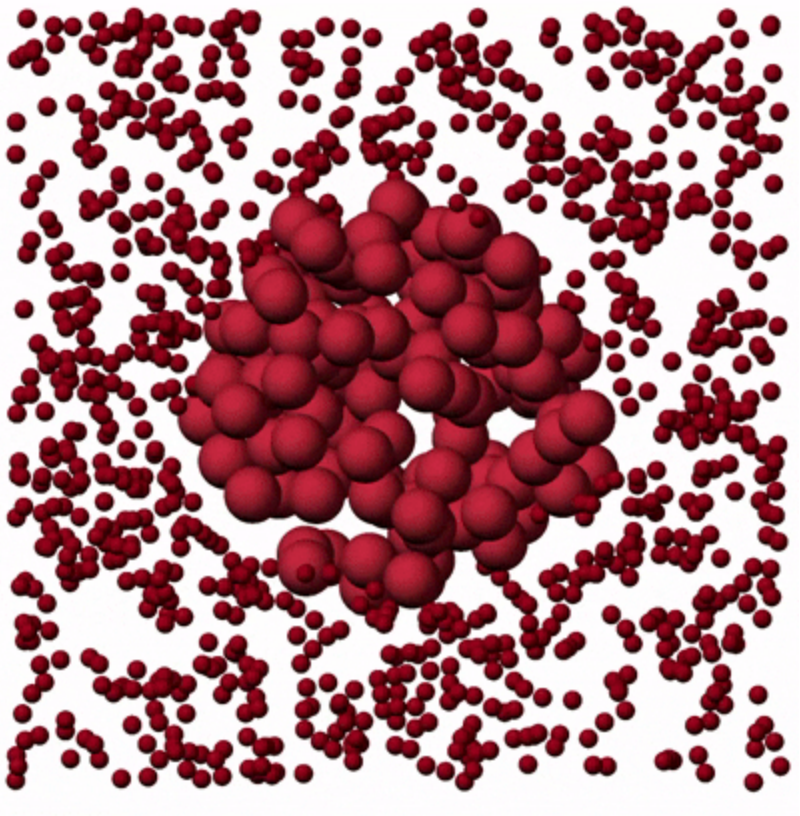
```
velocity all create T 49284 mom yes dist
gaussian
fix mylgv all langevin T T 0.1 10917 zero
yes
```

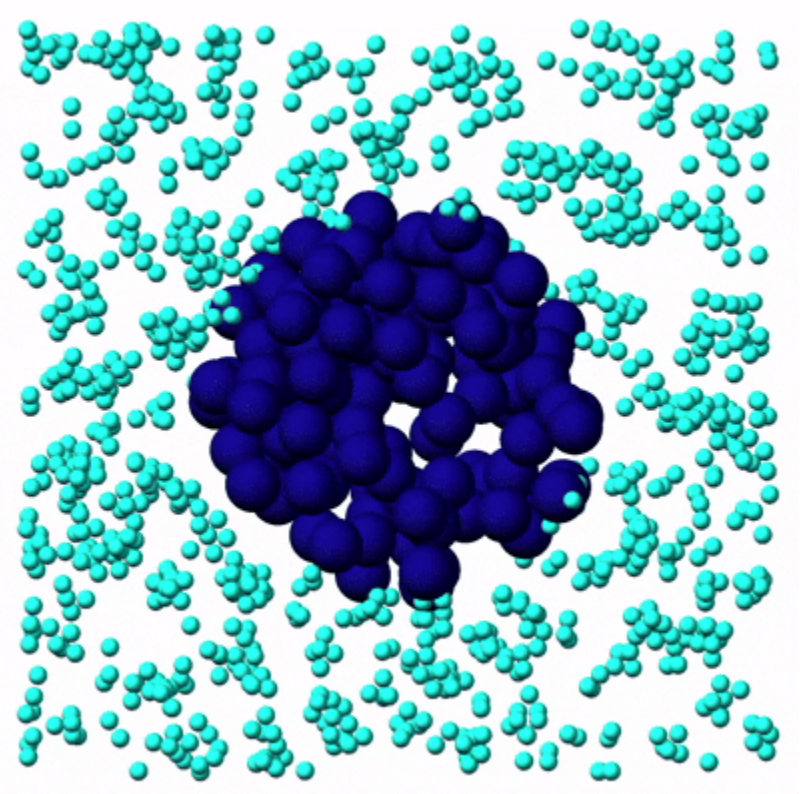
What it will show

Low T → ordered, slow, almost frozen

High T → chaotic, fast, strongly fluctuating

For visualization, I included two GIFs: one showing the system at a higher temperature (red colors), and one at a lower temperature (blue colors). These will be placed next to each other for comparison.





Interaction strength

This defines how atoms feel each other, attraction vs repulsion, weak vs strong bonding. Changing this can completely alter the structure of the system even at the same temperature.

What to touch (“ε” below)

```
pair_coeff 1 1 ε σ
pair_coeff 2 2 ε σ
```

What it shows

Stronger ε → clustering, tighter packing
Weaker ε → more gas-like behavior
This is basically changing the material itself.

There are many other parameters in the code that can be adjusted to explore different behaviors, such as the number of particles, masses, and the simulation box size. By carefully experimenting with the adjustable parameters, we can gain a deeper understanding of how the system evolves under different conditions.

Conclusion

As we have seen throughout our small journey with LAMMPS, molecular dynamics is a microscopic window into physical reality. It tracks individual atoms and their interactions and allows us to watch thermal agitation, phase changes, and structural rearrangements unfold in time. An application example from physics would be the use of MD and LAMMPS to simulate shock wave propagation in materials, where researchers were able to reproduce the formation and speed of shock fronts in crystalline solids under impact. These simulations helped make the connection between atomistic behavior and macroscopic shock properties measured in experiments, giving us insights that were inaccessible before.

In this work, I intentionally kept the language intuitive so that students really see what happens as atoms move, exchange energy, and reach equilibrium. My goal was to build a bridge from theory to practice without burying beginners in jargon. I hope this guide serves fellow students, whether bachelor or master level, who are just starting with molecular dynamics and want a friendly mental map of how MD and LAMMPS work in practice.

I’m Samah, a computational physicist who loves sharing knowledge and learning alongside others. If this guide helps you gain confidence and curiosity in MD, then it has fulfilled its purpose.

Thank you.

References & Acknowledgments

LAMMPS – Large-scale Atomic/Molecular Massively Parallel Simulator. https://www.lammps.org/

LAMMPS Tutorials. (n.d.). *Tutorial 1: Lennard-Jones fluid*. https://lammpstutorials.github.io/sphinx/build/html/tutorial1/lennard-jones-fluid.html

Gravelle, S., Alvares, C. M. S., Gissinger, J. R., & Kohlmeyer, A. (2025). *A Set of Tutorials for the LAMMPS Simulation Package* (LiveCoMS, 6(1), 3037). https://doi.org/10.33011/livecoms.6.1.3037

LAMMPS-GUI. (2025). https://arxiv.org/abs/2503.14020

Lee, J. G. (2011). *Computational Materials Science: An Introduction*.

Acknowledgement: I sincerely thank Prof. Abdelkrim Merad (University of Tlemcen) for his

course on Molecular Dynamics, which inspired this work.